week1 10/19-10/22 总结

工作内容汇总

产出

code\perprocessing.ipynb

项目理解

> 本项目要求根据已有数据，利用机器学习模型，预测信贷客户违约风险。
>
> 结合本次项目的数据字典，业务流程分为以下4个阶段：

阶段	input	内容
用户填写信息	purpose applicationTypetitleemploymentTitle employmentLength annualIncome homeOwnershippostCoderegionCode	通过地区、职业、资产获取客户基础画像
历史数据提取	delinquency_2years dti openAcc pubRec pubRecBankruptcies revolBal revolUtil earliesCreditLine n0-n14	交叉核验客户信息，保证信息准确性; 提取逾期记录、信用分数、负债、账户数量、公共负面记录等风险指标，作为评分模型输入
放贷结果	verificationStatus ficoRangeLow ficoRangeHigh	量化风险

阶段	input	内容
	grade subGrade loanAmnt term	
还贷结果	isDefault	目标

数据探索

1. 数据整体右偏，正负样本分布相同
2. 匿名变量大概率为过去信贷行为的统计量

沉淀思考-EDA流程

业务思考->提出假设->数据概览->数据类型转变->脏数据处理->缺失值、异常值、重复值处理->探索变量之间的关系->验证假设

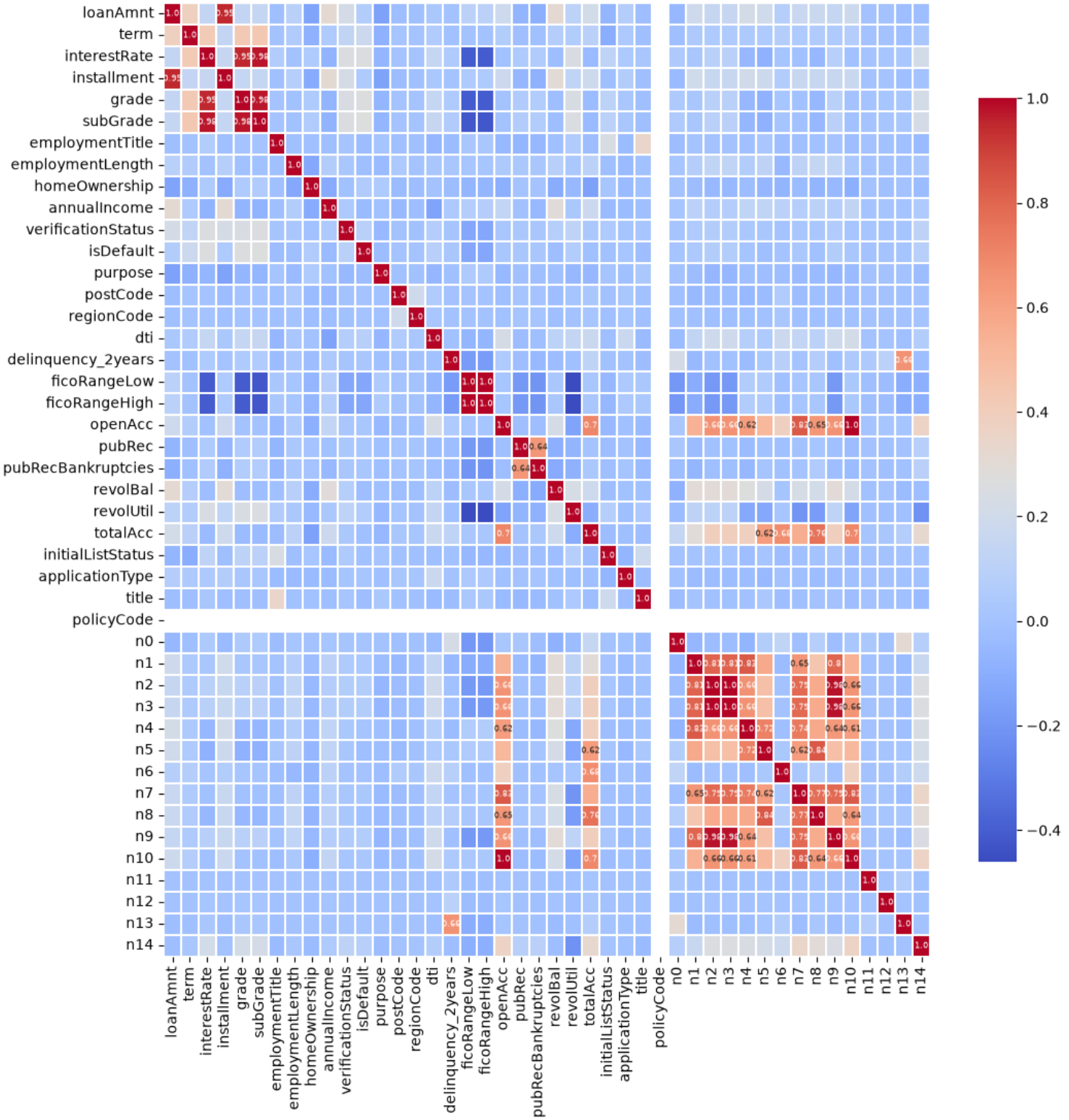
1. 数据内存管理方便后续特征工程
2. 数据类型转变结合真实语境
3. 金融项目中缺失值本身代表着特征，可以暂时不填充，添加缺失标记即可
4. 异常值从两个方面思考：先是逻辑上的异常（通常为取值范围，交叉核验），再是判断真正的异常与否。

卡点与下一步计划

1. 一开始我认为EDA与特征工程之间有严格的区分，后续发现两者需要相互迭代，不断修改完善。计划下周进行特征工程的时候改善当前EDA报告。
2. EDA流程应该以业务逻辑为主线，统计原则为辅助。没有普遍适用的规则，必须结合特定的语境，自己判断处理方式。例如，在处理异常值时，比较普遍的方式是3σ，此项目不符合正态分布，无法使用。又例如，缺失值本身代表着一种特征，在项目背景下，贷款人往往试图隐瞒个人信息获得贷款，很可能影响目标违约概率，不能轻易舍弃。整个EDA过程围绕着数据本身的分布展开，对业务目标的探索不足，后续应该事先提出假设，然后再验证，特别注意变量之间的关系为后续特征工程铺垫。

3. python代码不熟练，即使本周工作在ai辅助下大致完成，然而有许多想要实现的功能受限于代码能力暂时搁置。后续过程中，先完成，后完善，不断优化迭代。
4. 项目过程中沟通不足，后续定时汇报总结，及时调整方向。

.dataframe tbody tr th:only-of-type { vertical-align: middle; } <

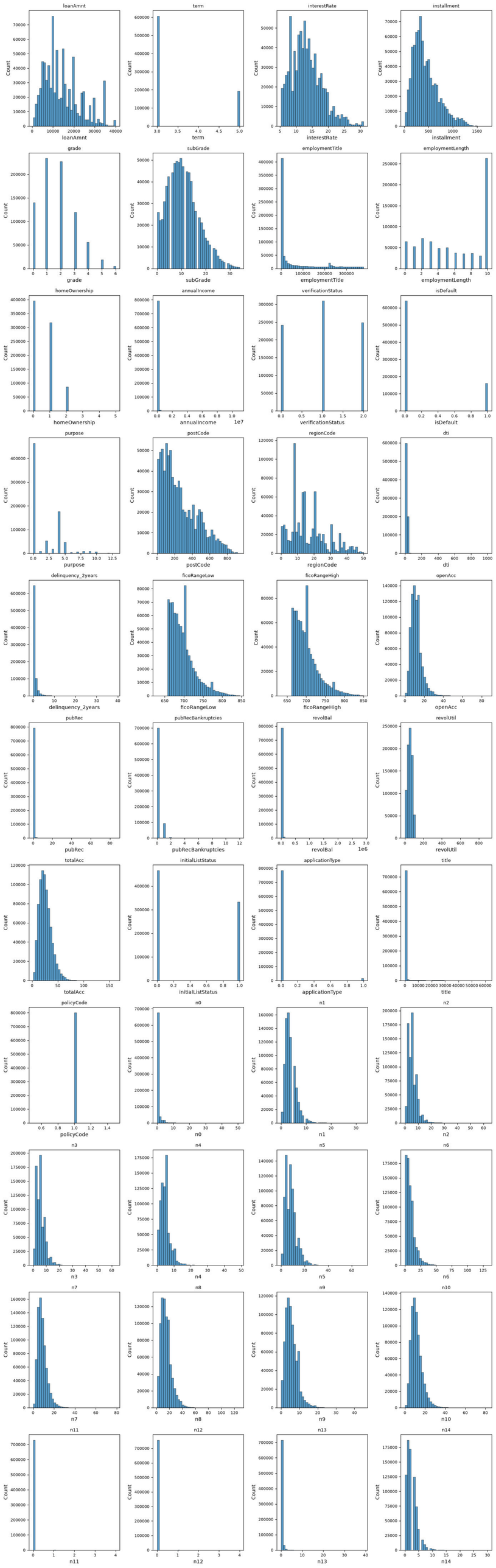


```python
continuous_columns = ["annualIncome", "dti", "revolBal", "revolUtil", "loanAmnt", "installment",
"interestRate"]

def check_anomalies(data):
    negative_cols = ["annualIncome", "loanAmnt", "dti", "openAcc", "totalAcc", "pubRec", "pubRecBankruptcies",
    "delinquency_2years"]
    business_flags = pd.DataFrame(index=data.index)
    rows = []
    for col in dict.fromkeys(negative_cols + continuous_columns):
        values = data[col].replace([np.inf, -np.inf], np.nan).dropna()
        q1, q3 = values.quantile([0.25, 0.75])
        low, high = q1 - 1.5 * (q3 - q1), q3 + 1.5 * (q3 - q1)
        business_flags[f"{col}_negative"] = data[col] < 0
        business_flags[f"{col}_infinite"] = np.isinf(data[col])
        business_flags[f"{col}_iqr_low"] = data[col].replace([np.inf, -np.inf], np.nan) < low
        business_flags[f"{col}_iqr_high"] = data[col].replace([np.inf, -np.inf], np.nan) > high
        rows.append({
            "字段": col, "最小值": values.min(), "最大值": values.max(),
            "最大值大于最小值": values.max() > values.min(),
            "负值数": int(business_flags[f"{col}_negative"].sum()),
            "极小值数": int(business_flags[f"{col}_iqr_low"].sum()),
            "极大值数": int(business_flags[f"{col}_iqr_high"].sum()),
            "无穷值数": int(business_flags[f"{col}_infinite"].sum()),
            "超过100数": int((data[col] > 100).sum()) if col == "revolUtil" else 0,
        })
    business_flags["dti_999"] = data["dti"] == 999
    business_flags["revolUtil_above_100"] = data["revolUtil"] > 100
    return pd.DataFrame(rows), business_flags
```

✓ 0.7s Python

	字段	最小值	最大值	最大值大于最小值	负值数	极小值数	极大值数	无穷值数	超过100数
0	annualIncome	0.00	10999200.00	True	0	0	39014	0	0
1	loanAmnt	500.00	40000.00	True	0	0	4246	0	0
2	dti	-1.00	999.00	True	2	0	3265	0	0
3	openAcc	0.00	86.00	True	0	0	27439	0	0
4	totalAcc	2.00	162.00	True	0	0	13446	0	0
5	pubRec	0.00	86.00	True	0	0	135235	0	0
6	pubRecBankruptcies	0.00	12.00	True	0	0	99519	0	0
7	delinquency_2years	0.00	39.00	True	0	0	154285	0	0
8	revolBal	0.00	2904836.00	True	0	0	47355	0	0
9	revolUtil	0.00	892.30	True	0	0	47	0	2719
10	installment	15.69	1715.42	True	0	0	24992	0	0
11	interestRate	5.31	30.99	True	0	0	14803	0	0


loanAmnt
term
interestRate
installment
grade
subGrade
employmentTitle
employmentLength
homeOwnership
annualIncome
verificationStatus
isDefault
purpose
postCode
regionCode
dti
delinquency_2years
ficoRangeLow
ficoRangeHigh
openAcc
pubRec
pubRecBankruptcies
revolBal
revolUtil
totalAcc
initialListStatus
applicationType
title
policyCode
n0
n1
n2
n3
n4
n5
n6
n7
n8
n9
n10
n11
n12
n13
n14
Count